

RECUT: provenance-aware recovery of persistent LLM state

Working paper and thesis decision brief

Prepared for Nazish Baliyan, 16 September 2026. For discussion with Prof. Abdulrahman Mahmoud. Authorship and submission require human review. This is a completed bounded pilot and a research proposal, not a completed thesis or an accepted/submission-ready paper. This final assessment supersedes the first two PDFs' prospective novelty/readiness recommendations.

Abstract

A numerical error in an LLM's KV cache can influence later cache entries and generated tokens before it is detected. Restoring the original location alone need not restore the computation. We study a conservative recovery protocol that reuses saved hidden inputs below a known faulty layer only while the corrected token history still agrees with the speculative history. At the first changed output, it discards future state and regenerates from the embedding layer. The protocol assumes a trusted checkpoint, localized single-layer corruption and deterministic recovery. Primary localization is an oracle; a separate snapshot-difference check tests non-oracle localization for this restricted persistent-prefix fault family.

On an RTX 5000 Ada GPU, the frozen primary study exercised 216 scenarios across SmolLM2-135M and Qwen2.5-0.5B: 192 finite perturbations and 24 no-op controls. Full replay, sparse RECUT and all-layer activation recovery each matched the clean reference exactly on every tested state/output/continuation check. Only 2 perturbed trajectories changed a greedy token within the W-step window, both at its final step. Against the conservative full-replay implementation, median paired sparse recovery speedups were 1.30x and 1.31x, with measured median normal-path overheads of 0.35% and 0.32%, respectively. In the separate all-case sensitivity study, sparse recovery ratios against the stronger checkpoint-alias full replay were 1.29x and 1.30x (above 1 means faster). Snapshot comparison localized 192/192 actually changed old-prefix cases; its broader blind spots remain unaddressed. Recovery speedup is not an end-to-end serving-throughput claim.

The findings support a narrow implementation result, not A/A*-level readiness. Activation-assisted KV reconstruction, delayed-error checkpointing and history-conditioned state reuse in self-speculative decoding are established. A late check of LayerSkip further narrowed the novelty claim. The proposed performance-thesis claim would need a distinct fault-specific result and a useful improvement over optimized adaptations with real verification and output-commit costs.

Plain-language outcome

An LLM's working memory can contain the consequences of an earlier mistake even after the original mistake is fixed. RECUT asks which intermediate calculations are still safe to reuse. It repairs the affected later calculations, but starts over through all layers once a corrected word changes the subsequent input. This restores the model's own fault-free computation; it does not make the model's answers factually true or aligned.

Decision for the professor

Discuss as a conditional investigation; do not commit to it as an A/A*-targeted thesis yet. The supervisor match is dependable AI execution, selective protection and fault analysis. The main unresolved issue is whether any fault-specific contribution is consequential rather than a straightforward composition of established ideas. The pilot is complete, but the requested high-impact publication goal is not established.

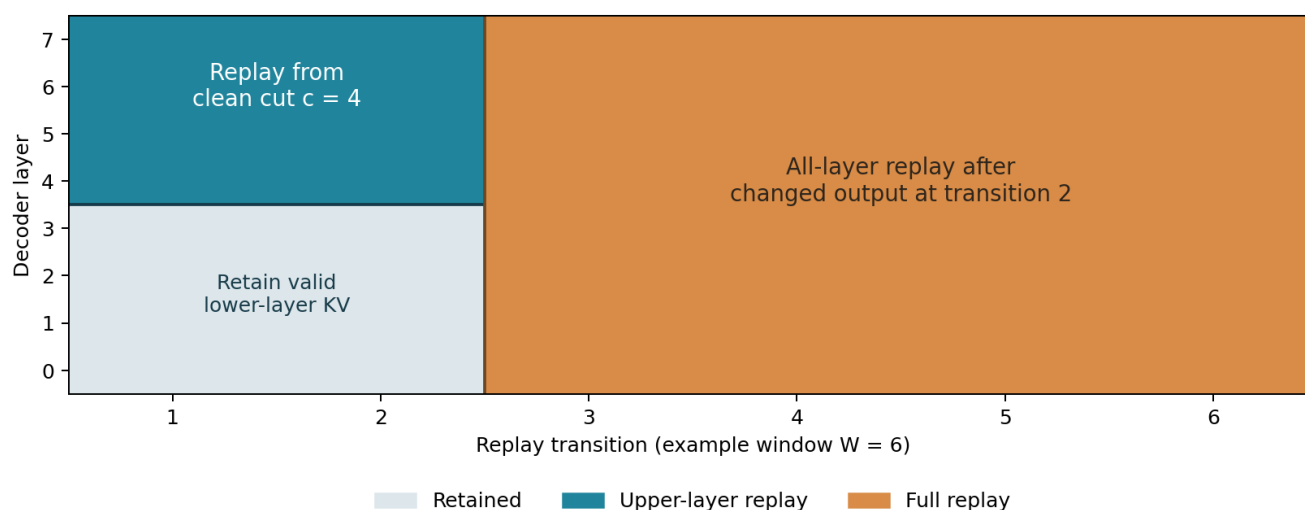
2. Problem, importance and supervisor fit

What existing repair can miss

The KV cache stores per-layer information from previously processed tokens. A finite error in an old entry may alter attention, later-layer inputs and future decisions. Some later cached values can be wrong even when the visible tokens have not changed. Conversely, a last-layer error can change the pending output without changing newly stored KV at that step. Both state and output must therefore be checked.

This distinction matters for persistent inference sessions, but the present study does not measure incident frequency in a production service. Cheap before-use integrity checks may be a better solution. A useful research result must establish when delayed recovery is needed and when its extra cost is justified.

Conceptual example: token divergence limits safe reuse



The figure is conceptual, not a measured trial: after the second repaired decision changes, every later step uses full-layer replay. Before that boundary, the lower-layer state can remain valid under the stated fault model.

Why it matches Mahmoud's background

The official MBZUAI profile describes reliable, high-performance AI systems, including hardware faults and software errors. His publication list connects application-visible fault effects, selective protection and numerical formats: GoldenEye (DSN 2022), selective CNN protection (ISSRE 2021), Minotaur (ASPLOS 2019) and related resilience work. RECUT is a direct methodological fit to that dependable-execution line. It is not evidence of his endorsement or of an unpublished research agenda. [Official profile](#); [publications](#).

The project is about numerical execution reliability, not a general solution to deception, jailbreaks or alignment. The AI-safety websites informed evaluation standards and discovery of nearby work; they do not make this automatically an alignment contribution.

3. Literature review and novelty boundary

Review coverage

All 100 supplied PDFs have a per-paper review entry and a matching nonempty extracted text. The collection has 1,767 physical-page markers. Eighteen selected papers received deeper focused review; 82 received selected-section screening. This is not 100 exhaustive cover-to-cover reads or independent reproduction of their results. The readable 100-paper ledger and three source ledgers accompany this draft.

All 30 requested websites were checked, with SPAR and MATS as two extra resources. The access audit distinguishes readable project pages, navigation-only results, unavailable rosters and failed links. Program inclusion is neither technical evidence nor institutional endorsement. Two supplied records, 036 and 039, have unresolved identifier/date inconsistencies; neither is essential to this study's central claim.

Closest work	What must not be claimed as new	What this pilot actually adds
HCache; HybridServe; KVPR	Exact KV reconstruction from activations and storage/recomputation tradeoffs.	An executable delayed-corruption recovery contract with a token-divergence boundary; no superiority over their full implementations is established.
LayerSkip; VeriCache	History-conditioned verification and rollback, including layer-state reuse or lossy-KV drafts.	A localized numerical-fault/checkpoint contract, not a new general state-reuse or exact-output correction principle.
Classical delayed-SDC checkpointing	Verification delay, retaining clean checkpoints and checkpoint-period optimization.	A transformer layer/token instantiation, not a new checkpointing principle.
vLLM valid-prefix recovery RFC	Selective rollback of affected requests to a valid prefix.	Comparison against a full suffix-replay implementation with the same trusted checkpoint.
Shared-KV fault study; Models Take Notes	Persistence, propagation and stale derived cache information.	State/logit/token controls and exact recovery within the stated numerical path; not first discovery of the phenomenon.
GhostServe; Concordia	KV redundancy, partial recomputation, snapshots and logs; GhostServe's accessed discussion includes soft memory errors.	Controlled delayed trajectory-repair tests, not an implementation or demonstrated improvement over those systems.

3. Updated novelty assessment (continued)

The remaining candidate contribution in this study is a fault-specific recovery contract and a consequential, measured systems result. The general token-history validity rule is not a sufficient novelty claim: LayerSkip already combines activation/KV reuse with acceptance of a matching prefix and rollback at disagreement. It does not study this delayed numerical-fault contract, but it substantially strengthens the incremental-composition objection. The first two PDFs preserve the earlier prospective assessment; this late finding narrows it rather than being silently omitted. [LayerSkip, sections 4.3 and Appendix A.4](#).

The acronym, activation journals, output buffering, a checksum and error propagation are not new contributions. The correctness argument is a standard induction, not a demonstrated optimality theorem.

VeriCache also checks compressed-KV drafts against full KV and corrects the first disagreement. Exact-output correction after altered KV is therefore not an empty prior-art category. The final six-record falsification note separates each accessed method from this pilot's narrower fault contract. [VeriCache, section 4.1](#).

The strongest skeptical objection remains valid: this may be an obvious application of checkpointing to a known dependency graph. No accessed source was found to establish this exact tested combination, but a search cannot prove priority. The pilot justifies further falsification, not a statement that the idea is already high-impact or publication-ready.

4. Recovery contract and algorithm

Let N be the number of layers, indexed 0 to $N-1$. A clean prefix has T processed tokens. A perturbation affects existing KV at layer L immediately after a trusted checkpoint. The next input token was selected before the fault. The model then executes W uncommitted transitions. A journal cut c is the hidden input immediately before layer c , not its output.

Required assumptions

Fixed weights; evaluation mode; one unpadded sequence; ordinary append-only, layer-local unquantized KV; default rotary positions; fixed eager single-token arithmetic; deterministic greedy decisions; and trusted checkpoint, journal payloads, localization and recovery computation. No sampling, concurrent batching, recurrent/hybrid state or real external action is included.

The first pilot has oracle localization and a known clean time boundary. Metadata checks do not certify a journal's integrity. A corrupt checkpoint or incorrectly reported layer can invalidate the guarantee. Unsupported cases are rejected; this is not an operational safety controller.

Procedure

1. Keep W decisions private. Save hidden inputs at selected cuts, with position, input token and engine identity. Preserve a trusted prefix checkpoint available to every valid method.
2. Choose the deepest saved $c \leq L$; $c=0$ uses ordinary full-layer replay. Restore the checkpoint in the layers to be replayed. Skipped lower layers may retain a speculative suffix only while the input histories agree.
3. Replay each transition from its clean journal input. If the corrected output first differs at one-based transition j , crop every layer to exactly $T+j$ processed tokens and permanently disable old journals. Transition j itself is still cut-replayable; transition $j+1$ must use the corrected token from the embedding layer.
4. Do not re-enable journals if token streams later reconverge. At the final transition, keep the corrected pending output even though it is not yet represented in KV. The experiment validates the complete state and a two-step continuation against a separate clean run outside the recovery timer.

Conditional correctness and work

For $c \leq L$, lower layers cannot read the faulty higher-layer KV except through future token choices. While histories agree, their states and the saved layer- c input remain clean. Replaying the upper region from the clean prefix therefore reconstructs its outputs by induction. At divergence, cropping establishes a completely clean frontier; ordinary decoding reconstructs the remaining suffix.

With $j=\text{infinity}$ when outputs never diverge and $k=\min(j,W)$, executed layer-token work is $Q = N W - c k$. This is a count of work, not a wall-time prediction or proof of minimal repair. Byte-identical execution is not claimed: checks use exact elementwise tensor equality under fixed arithmetic, which does not distinguish signed zero.

5. Implementation and experimental protocol

Executed scope

The primary run used one NVIDIA RTX 5000 Ada Generation, BF16, batch size one and eager attention. The pinned runtime was Python 3.13.9, PyTorch 2.6.0+cu124, Transformers 4.51.3 and CUDA 12.4. Deterministic algorithms were enabled, TF32 disabled, and the execution configuration retained in metadata. The then-current 58-test CPU suite passed in the allocated HPC environment before the primary run. The eight-case debug run used separate inputs and is excluded from every primary result below.

The implementation calls the installed Hugging Face decoder layers, attention-mask helper, rotary module and output head. It does not hand-reimplement attention math. Every native-parity gate checks both logits and complete KV against the library's full incremental forward. The primary gate spans 128 prefix positions plus 14 further steps per model, covering the maximum $W=12$ window and two-step continuation.

Frozen matrix

Four constructed prompts cover prose, arithmetic, structured records and code-like text. Tokenized inputs are repeated/truncated to exactly 64 or 128 tokens; these are controlled mechanism workloads, not a standard task-quality benchmark. The same prompt may appear in multiple trials, so repeated cases do not establish broad language-task generalization.

Per model: 4 prompts x 3 layer fractions x 2 windows x 2 cache kinds x 2 perturbation shapes = 96 perturbed cases, plus 12 no-op controls. Layers are $\text{floor}(\text{fraction} \times (N-1))$ for fractions 0, 0.5 and 1. Windows are 4 and 12 transitions. K and V are both tested. Separate debug prompts/seeds are excluded from primary counts.

For each perturbed case, a seeded CPU generator chooses a prefix token, head and coordinate without searching model outputs. A scalar or complete head vector receives Gaussian additive noise scaled by eight times that layer tensor's RMS, then rounds to BF16. These stress perturbations are synthetic; they do not estimate physical bit-flip distributions or field fault rates. Before/after values and actual changed-element counts are recorded.

Methods and measurement

No recovery and original-slot restoration are negative controls. Full suffix replay is the main valid baseline. Sparse RECUT journals at $\text{floor}(N/4)$, $\text{floor}(N/2)$ and $\text{floor}(3N/4)$; the all-layer baseline journals every nonzero cut and chooses $c=L$. It is an adaptation of activation-assisted recovery, not a reproduction of HCache or HybridServe.

Each valid method has one excluded warmup and three counterbalanced timing repetitions. Restoration and replay are timed with GPU synchronization; common preparation of the working cache and experimental correctness diagnostics are excluded equally. Fault-free overhead uses actual no-journal/sparse/all-layer acquisition, with warmup and five rotated repetitions. Snapshot copy times, payload bytes and fixture-inclusive peak allocations are retained separately.

6. Correctness results

Method / endpoint	Perturbed cases	No-op controls
No recovery: all checks equal	0/192	24/24
Original-slot restore: all checks equal	1/192	24/24
Full suffix replay: all checks equal	192/192	24/24
Sparse RECUT: all checks equal	192/192	24/24
All-layer journal: all checks equal	192/192	24/24
No recovery: tokens only	190/192	Not the full-state criterion
Original-slot restore: tokens only	190/192	Not the full-state criterion

Every valid recovery method passed 108/108 cases on SmolLM2 and 108/108 on Qwen. SmolLM2-135M: 0/96 token-changing faults; Qwen2.5-0.5B: 2/96 token-changing faults. Original-slot restoration matched the cache in 64/192 faults and final logits in 1/192; these narrower endpoints do not replace the joint recovery criterion. There were 0 perturbed cases with no actual rounded tensor change. These counts are descriptive of the fixed matrix, not estimates of service fault impact. Multiple recovery policies evaluated on the same corrupted trajectory do not multiply the sample size. Both token divergences occurred at $j=W=12$: Qwen primary-052 at layer 0/cut 0 and primary-063 at layer 11/sparse cut 6. Thus no primary case executed a replay step after an observed token divergence; the primary alone does not empirically validate that branch. The faulty/no-repair state, logits or continuation were non-finite in 0/192 cases. After slot-only restoration, 126 cases kept the same W tokens but had a different cache; 63 had the same cache but different final logits. Thus not every failed repair is descendant KV corruption. The two additional uncorrected continuation decisions differed in 5/192 cases; the two within-window divergences are not the count of all observed downstream token effects. These are numerical-state findings, not measured harmful-content outcomes.

6. Correctness evidence (continued)

Checks beyond matching the generated words

Every valid repair is compared on all K/V tensors, shapes, dtypes, finiteness, logical cache counters, W output decisions and final logits. A two-step continuation checks subsequent tokens, cache and logits. The final predicted token is explicitly treated as pending, not already cached. Per-repetition traces verify the layer-work identity; primary branch coverage is qualified below.

A separately declared, post-hoc branch test reused Qwen primary-063's exact prefix, seed and recorded fault, extending W from 12 to 14 without searching alternative faults. Its first 12 clean/faulty decisions and complete injected-fault record matched the primary. At output 12 the histories diverged; sparse replay used cut 6 for transitions 1-12 and layer 0 for 13-14. All three valid methods exactly recovered the extended state and continuation. Sparse/all/full decoder-layer work was 264/204/336, respectively. A separate CPU audit passed 1,194 checks including the saved 8.08 MiB state/logit/token archive and traces. This is one selected branch diagnostic, not an additional independent fault or a performance result. Archived tensors omit continuation; its equality remains live runner evidence.

A separate audit program passed 135,764 evidence-consistency checks with 0 failures in the pinned HPC CPU runtime. It checked all 216 rows, 1,944 repair timing samples, 3,240 normal-path timing samples, and 3 selected complete-state snapshot files (26.39 MiB). Each model's native-forward gate covered 142 positions. The auditor independently recomputed summary statistics and checked stored state tensors using weights-only CPU loading. Snapshots do not contain continuation tensors: those checks are recorded runner evidence, not independently rechecked snapshot tensors. The original-runtime audit reported no advisory warnings. The second audit on macOS retained an explicitly recorded cross-runtime advisory; it does not replace the original-runtime result.

What zero observed failures means

It establishes success on these recorded deterministic scenarios. It does not prove zero failure probability for other prompts, fault families, models, precision paths or deployments. Timing repetitions and three methods on the same injected state are not additional independent fault trials. Even a textbook zero-failure binomial bound would require an IID sampling claim that this constructed workload does not provide.

The clean reference is an experimental oracle sharing the model's underlying operations. Native-library parity and analytic counterexamples reduce shared-implementation risk, but this is not machine-checked verification or external replication. The distinct audit program checks evidence consistency and stored snapshots; it is not an independent laboratory rerun.

7. Recovery cost, normal cost and memory

Model / method	Full-replay speedup: median [IQR]	Normal overhead: median [IQR]
SmolLM2-135M / sparse	1.30x [1.01, 3.46]	0.35% [-0.22, 0.95]
SmolLM2-135M / all-layer	1.82x [1.01, 17.88]	1.91% [1.32, 2.43]
Qwen2.5-0.5B / sparse	1.31x [1.00, 3.46]	0.32% [-0.41, 1.06]
Qwen2.5-0.5B / all-layer	1.77x [1.01, 11.63]	2.05% [1.34, 2.70]

Each entry is computed across 96 perturbed cases per model using paired per-case medians, not a ratio of pooled medians. Median conservative full-replay times were SmolLM2-135M: 243.55 ms; Qwen2.5-0.5B: 206.26 ms. IQR denotes the 25th-75th percentiles, not a confidence interval. Five normal-path rotations across three modes do not completely balance every order position; the small observed overhead differences are noise-sensitive, not overhead guarantees. The original comparison combines reduced decoder work with fewer checkpoint-prefix copies; it must not attribute all speedup to the layer-work identity. The next section tests a stronger restoration baseline. The all-layer method is not hidden: it can recover faster while paying substantially more activation storage.

Model	Shared full checkpoint	Extra journal payload: sparse / all
SmolLM2-135M	1,440.0-2,880.0 KiB	13.5-40.5 KiB / 130.5-391.5 KiB
Qwen2.5-0.5B	768.0-1,536.0 KiB	21.0-63.0 KiB / 161.0-483.0 KiB

The payload figures exclude Python metadata, allocator fragmentation and protection of trusted state. Raw peak allocations include resident reference/faulty/checkpoint fixtures, so they must not be presented as the memory of a standalone serving deployment. Both models use grouped-query attention; a hidden activation is not universally half the size of that layer's KV.

Break-even is not an observed fault rate

If H is the added journal cost per window and S is saved recovery time per incident, a simple single-incident expected-time model needs $p > H/S$ when both are positive. The experiment estimates costs for its artificial case mix, not p . Verification and checkpoint costs common to the compared recovery methods cancel only within that restricted comparison; they still matter to a deployment. Private-window latency is a separate application cost.

SmolLM2-135M sparse: mean $H=0.817$ ms and $S=75.498$ ms; $H/S = 1.08\%$ of windows. Qwen2.5-0.5B sparse: mean $H=0.689$ ms and $S=63.931$ ms; $H/S = 1.08\%$ of windows. No observed deployment-rate is available to show that either threshold would be met. These primary ratios use conservative copy-and-replay, not the stronger alias baseline from a separate run. The companion exploratory frontier analysis retains every case and reports layer/window/fault-shape/divergence strata; its nonnegative-cost sensitivity is not a statistical bound or a confirmed Pareto frontier.

Any negative measured overhead is timing noise or an unresolved implementation effect, not proof that storing journals is free. Recovery speedup is not service-throughput speedup. The custom eager decoder is a correctness-oriented reference, not an optimized vLLM/FlashAttention baseline.

8. Stronger-baseline sensitivity

An independent implementation review found a potentially avoidable cost in the conservative full-replay baseline: restoring every prefix layer by copying checkpoint tensors. Because the contract gives a single known faulty layer, a stronger baseline can truncate all suffixes with views, restore only that layer's complete K/V prefix, and still replay every layer normally. It does not need the exact corrupted coordinate.

An even simpler baseline starts a fresh cache by aliasing the same trusted resident-GPU checkpoint, avoiding all prefix copies. Appends allocate new tensors. The implementation checks that checkpoint and original fault-state payloads remain unchanged. The alias baseline needs no fault-coordinate information. These views retain backing allocations; no physical memory release is implied.

This extension was specified while the frozen primary run was in progress, without changing that run. It repeats all 192 perturbed primary model/case pairs, not a selected favorable subset. Five rotated timing repetitions and a warmup compare conservative full replay, view/one-layer restore, checkpoint-alias replay, sparse RECUT and all-layer RECUT. These reused scenarios are not 192 additional independent faults. The supplementary run is explicitly post-hoc baseline sensitivity.

Method	Alias-full / method: SmoLM2	Alias-full / method: Qwen
Conservative full replay	1.00x [0.99, 1.00]	0.99x [0.99, 1.00]
View / one-layer restore	1.00x [0.99, 1.00]	1.00x [0.99, 1.00]
Checkpoint-alias full replay	1.00x [1.00, 1.00]	1.00x [1.00, 1.00]
Sparse RECUT	1.29x [1.01, 3.45]	1.30x [1.00, 3.43]
All-layer journal	1.83x [1.01, 17.93]	1.76x [1.00, 11.62]

Ratios are paired per-case medians with interquartile ranges. Above 1 means the listed method is faster than checkpoint-alias full replay; below 1 means slower. All 96 perturbed cases per model are included.

Every method recovered all 192 reused cases, with exact state/output/continuation checks and immutable-source guards in all 4,800 measured recovery trials. A separate evidence audit passed 98,997 consistency checks. It does not independently reload supplemental tensors: none were saved. The final expanded unit suite contains 76 passing tests, distinct from the original 58-test primary gate.

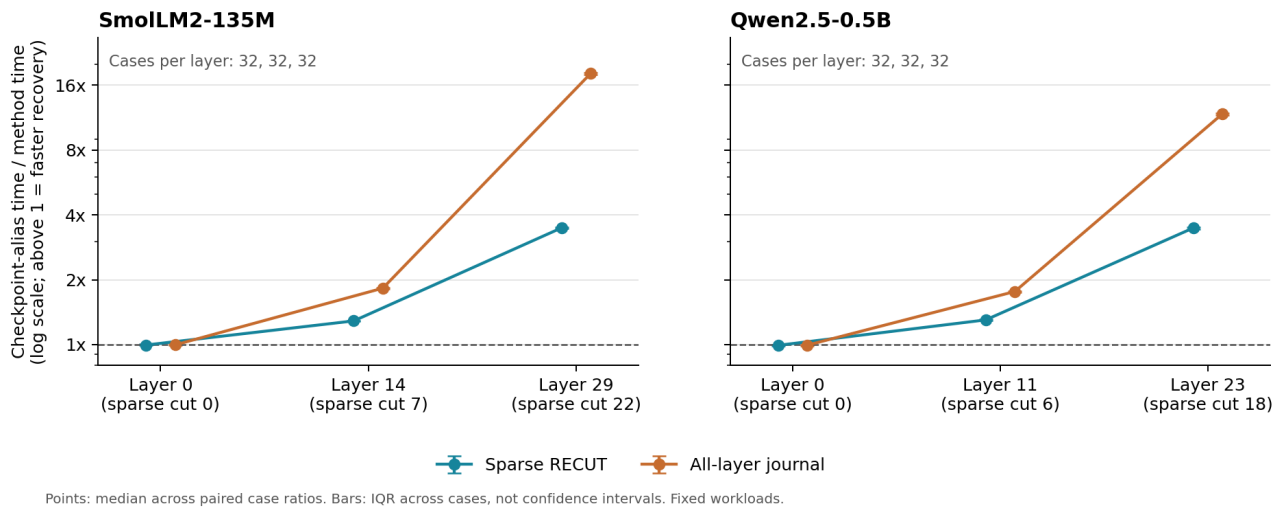
Median checkpoint-alias replay times were SmoLM2-135M: 239.42 ms; Qwen2.5-0.5B: 199.71 ms.

At layer zero, conservative full and sparse invoke the same replay algorithm. Their same-algorithm control ratios were SmoLM2-135M: 1.00x; Qwen2.5-0.5B: 1.00x. The alias-full/sparse ratios at that layer were SmoLM2-135M: 1.00x; Qwen2.5-0.5B: 0.99x, reflecting restoration differences as well as timing variability. These cases save no decoder-layer work. Per-layer/window strata remain in the summary; a pooled gain is not a gain at every layer.

Only within-supplement paired comparisons support conclusions about this implementation change. Comparing times from separate GPU runs would mix restoration changes with system variability. The optimized full baseline is still an eager, batch-one reference; it is not a production serving baseline. Sparse and all-layer methods retain their original restoration code, making this a stronger challenge to them rather than a joint optimization of all methods.

9. Where the recovery gain comes from

Observed recovery tradeoff by fault location



This figure uses only the separately audited supplemental run. Each point summarizes all cases at that tested layer; the same four prompts and two windows recur, so these are descriptive strata rather than independent workload samples. Error bars show interquartile ranges, not statistical confidence. No primary timing samples are mixed into the figure.

At the first layer, no lower-layer work can be skipped. Later clean cuts can avoid more work until a token decision changes. The all-layer method chooses cut L , whereas the sparse policy chooses the deepest configured cut no greater than L . The measured ratios include the policies' restoration and bookkeeping, not only decoder-layer invocations.

The sparse cuts were fixed independently of outcomes at $\text{floor}(N/4)$, $\text{floor}(N/2)$ and $\text{floor}(3N/4)$. The tested middle fault is $\text{floor}((N-1)/2)$, just below the middle cut, so it selects the quarter-layer cut. Consequently the middle configured journal is not selected by these three tested fault locations, although its collection cost is paid. No placement tuning was performed after observing results. These measurements do not establish the best cut placement or a globally optimal storage/recovery frontier.

Late-layer benefits must not be generalized to a workload dominated by early errors or early token divergence. No real fault-location distribution was measured. The formula $Q = N W - c \min(j, W)$ describes the supported implementation's work; it does not make the balanced test mixture a deployment model.

10. Restricted detection feasibility

The existing trusted checkpoint makes a simple comparison possible. Old prefix entries are immutable under the supported append-only decoder. At the end of the window, the detector compares those entries with the checkpoint, reduces mismatches per layer on the GPU, and transfers the resulting layer flags to the host. Its input does not include the injected layer or coordinate. A unique changed layer routes supplemental repair; no difference falls back conservatively, and multiple changed layers are outside this pilot's recovery contract.

This is ordinary snapshot-difference detection, not a new error-detection invention. It was added as a separate post-hoc check before supplemental measurements. The original primary run still uses oracle localization. All 216 primary model/case pairs are reused for detection, including 24 no-op controls; the supplemental recovery timing matrix contains only the 192 perturbed pairs.

Detector endpoint	SmolLM2-135M	Qwen2.5-0.5B
Actually changed prefix cases localized	96/96	96/96
No-op controls with an alarm	0/12	0/12
Comparison time, all cases: median [IQR]	0.930 ms [0.852, 0.999]	0.727 ms [0.678, 0.801]
Logical comparison reads per window	2.81-5.62 MiB	1.50-3.00 MiB

The comparison localized 192/192 actually changed prefix cases and raised 0 alarms on 24 no-op controls. There were 0 intended perturbations with no actual rounded change. These are controlled-case results, not sensitivity/specificity estimates for hardware faults. All 1,080 recorded detection timings are separate from recovery timings; no new normal-journal overhead experiment was run.

The supplemental repair policy used the detector's unique layer result, not the injected layer. Ground truth was checked only to evaluate the controlled experiment. Exact comparison consumes the already-required trusted snapshot and at least two copies of its logical prefix payload in reads; the table does not count all temporary-flag traffic or protected-storage cost. No claim of low-overhead deployment follows from these short-context measurements.

Why coverage remains narrow

The comparison scans the old prefix, not the newly generated suffix. It cannot detect suffix-only corruption, compute-only errors that leave the compared prefix unchanged, a transient error restored before checking, weight or journal errors, a bad checkpoint, or matching damage to both copies. Unit controls deliberately expose suffix-only and restored-transient blind spots. No-op false alarms and detection of this injected family are not population error rates.

The full snapshot must still be stored, updated and protected. Logical payload reads are not measured HBM traffic; allocations, synchronization and long-context scaling also matter. Detection is measured outside the recovery timer and is common to the compared repair choices. The study has no network commit protocol, live clients or proof that all committed computation is fault free. In particular, checking only a prefix cannot certify that a newly created checkpoint is clean.

11. Validity limits and publication assessment

Limits that cannot be hidden by a good plot

Detection and trust: primary localization is supplied by the injection harness. Supplemental snapshot comparison localizes only the persistent old-prefix faults covered by its contract. There is no measured general fault detector, population false-negative rate, uncertain-localization recovery or trusted-storage failure model. Errors before the checkpoint, faults in model weights or journals, multiple-layer faults and faults during repair are outside scope.

Fault realism: finite source-level tensor perturbations are not physical fault induction or validated instruction-level error profiles. No NVBit or GPU binary instrumentation was used; lab permission for that remains unresolved. No security exploit or external target was exercised.

Scale and serving: two compact models, short contexts, one GPU, batch size one, eager attention and four constructed prompts are insufficient for production or broad quality claims. The Qwen library emitted an eager/sliding-window warning; its configuration disables sliding-window use, contexts remain short, and native numerical parity is checked. No general sliding-window support is claimed.

Branch coverage: only two of 192 primary perturbations changed greedy tokens within the W-step window, both at its last step. The separately selected two-token extension probes one observed boundary; it cannot establish general recovery behavior after long or diverse token divergences.

Baselines: the supplement measures two copy-avoiding full-replay variants. Further fused recovery, jointly optimized journal repair, production batching, before-use verification and published systems' full implementations may change the frontier. The all-layer baseline is a strong comparator within this replay family, not a universal optimum.

Commitment: buffer ownership is modeled by private Python data, not a network service with real clients. Already-emitted wrong text or external actions cannot be undone by repairing KV. The study measures window computation cost, not a production output-commit SLO.

11. Publication assessment (continued)

Present decision

The predeclared correctness gate passed on the supported frozen matrix. The 1.2x recovery-only median screening signal against the original baseline was met for both models. The sparse median recovery-only signal also survived checkpoint-alias full replay on both models. Neither result establishes the predeclared performance-thesis gate. Only the restricted snapshot-comparison cost was measured; broader end-to-end detection/commit economics, a production baseline, an acceptable holdback budget and a material advance over optimized prior-work adaptations remain unestablished. The late self-speculative-decoding comparison also leaves the novelty gate unmet; the general reuse-and-rollback rule is already known. The defensible status is a completed, auditable pilot worth a skeptical supervisor discussion, not a selected perfect thesis, a demonstrated high-impact contribution or an A/A*-ready submission.

DSN and ISSRE are A-ranked in ICORE2026; ASPLOS is A*. The topic is closest to dependable computing. ASPLOS requires a substantive architecture/OS/PL advance, not simply running an LLM on a GPU. Rank and topical fit are not acceptance predictions. [DSN rank](#), [ISSRE rank](#), [ASPLOS rank](#), [ASPLOS scope](#).

The official DSN 2027 call lists abstracts on 25 November 2026 and papers on 2 December 2026, AoE. That is a planning opportunity, not a promise this work can meet the standard or deadline. Recheck the selected track with the professor. [Official call](#).

12. Thesis roadmap and professor discussion

Three decision gates, in order

1. **Defeat the closest adapted baseline.** Compare optimized valid-prefix replay, activation restoration and self-speculative state-reuse principles adapted to the same fault contract and equal byte/commitment budgets. Include simpler verification before use. Establish a non-dominated operating region and a distinct fault-specific contribution or stop the performance-thesis claim. The work-count identity alone is not the contribution.
2. **Extend beyond snapshot-only localization.** Specify a realizable detector with an explicit trusted computing base and verification window for the actual target fault family. Charge read/write cost, snapshot validation, false alarms, misses, localization granularity and fallback. Test uncertain locations, suffix/compute-origin errors and out-of-window events. The simple prefix comparison is a feasibility control, not a solution to these requirements.
3. **Validate the surviving case in real serving.** Use a larger supported model, longer contexts and an actual task corpus in an optimized single-GPU path before attempting distributed scale. Preserve exact numerical controls, measure normal throughput and recovery tails, and expand fault families with a justified mapping to real error mechanisms. Architecture-level instrumentation requires lab authorization.

These are proposed extensions, not experiments already performed. The first gate should be an early stop/go test; adding more models without resolving novelty and detector economics is not sufficient.

Suggested opening in the meeting

"I screened the supplied 100-paper corpus and additional work. Activation-based KV restoration, checkpointing and history-conditioned reuse already exist, so I am not claiming those as new. I built a bounded delayed-fault recovery study, checking full state and output rather than plausible answers. The implementation works in its tested scope, but a late LayerSkip comparison weakens the novelty case. I would like your judgment on what distinct fault-specific result could justify continuing, or whether this direction should be rejected as too incremental."

Decisions to obtain from the professor

Agree on the target fault/detection model, the strongest competing implementation, acceptable output holdback, and the minimum result that justifies a full thesis investment. Ask what additional mathematical or architecture contribution is expected by the degree and intended venue. Discuss access to realistic error traces or authorized fault-analysis tools; do not infer permission from possessing HPC credentials.

When to reject or redirect

Reject the claimed advantage if it depends on unavailable oracle information, omitted steady-state cost, selected late-layer faults only, or unacceptable buffering. A negative-result paper would need a general, novel explanation of a failure boundary, not just an unsuccessful prototype. No research direction is "perfect" before those questions are answered.

13. Artifact, reproducibility and disclosure

Primary episodes SHA-256: 6c2d4a577eb57bd7... (full digest in metadata and the artifact manifest). The primary configuration begins bd9673777665c488..., and engine source begins c0e0949ccf415853.... Primary Slurm job 195057 and the separate audit/sensitivity jobs are recorded with timestamps and logs in notes/execution_log.md. The README provides immutable model-fetch, test, run, analysis and audit instructions. The first two PDFs preserve the prospective idea and plan; this document reports measured outcomes and limitations. One plan wording is resolved conservatively: a missing journal raises an error (fail closed), rather than implementing automatic fallback. Normal-path window computation was measured, but an end-user/private-commit latency SLO was not.

The supporting package includes the source and pinned environment, immutable model revisions, frozen configs, raw JSONL episodes, native-parity traces, timing repetitions, selected complete-state snapshots, descriptive summary, separate audit, post-hoc boundary diagnostic, 100-paper review, website audits, proof sketch and all three PDFs. It excludes model weights, Python environments, SSH sockets and credentials. Earlier RAVEN/ProofOps results were not changed or reused as RECUT evidence.

The SSH connection used the supplied authorization. Computation and installations ran only within Slurm allocations. One initial setup failed because system Python lacked venv support; a user-space environment resolved it. Failed logs remain included. The scheduler accounting service was unavailable, so the record does not invent a sacct exit status. No AWS cluster changes, public deployments, external messages or conference submissions occurred.

AI assistance and responsibility

Literature screening, code, tests, analysis and this draft were produced with AI assistance, including separate-agent reviews. Those reviews are not human peer review or external replication. Nazish and the supervisor must verify references, scientific claims, authorship, institutional AI-use rules and the final manuscript before submission. This draft does not list the professor as a coauthor without agreement.

13. Selected primary references (continued)

1. [HCache](#), EuroSys 2025 author record; activation-assisted state restoration.
2. [HybridServe](#), ICCD 2025 author record; hybrid activation/KV caching.
3. [KVPR](#), 2025 revision; partial KV recomputation.
4. [Aupy et al., silent-error detection and checkpointing](#), PRDC 2013 author record. A later date printed in the fetched PDF is not treated as a new publication date.
5. [vLLM KV-load failure RFC](#), engineering proposal; not a peer-reviewed result or implementation verification.
6. [Shared-KV bit-flip study](#), 2026 author record; defensive high-level comparison only.
7. [Models Take Notes at Prefill](#) and [Stage-Replay Divergence](#), 2026 preprints.
8. [Aborted but Not Forgotten](#), 2026 preprint; rollback-consistency contract, no exploit reproduced.
9. [GhostServe](#), MLSys 2026 publisher record; KV redundancy and partial recomputation. Its [accessed author version](#) discusses soft memory errors. [Concordia](#), 2026 preprint; GPU checkpoint/log recovery.
10. [SmolLM2-135M](#) and [Qwen2.5-0.5B](#), official public model cards. Exact revisions are in the artifact.
11. [LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding](#), ACL 2024 publisher record; author-text sections 4.3 and Appendix A.4 are a direct mechanism-level novelty constraint, discovered in the final falsification pass.
12. [VeriCache: Turning Lossy KV Cache into Lossless LLM Inference](#), 2026 preprint; section 4.1 constrains any broad exact-output recovery claim. Reported performance was not independently reproduced.